# Nisha Amitbhai Patel

Fairfax, VA | (+1) 267 446 4419 | npatel61@gmu.edu | LinkedIn | GitHub

## Research Interests

Trustworthy and reliable retrieval-augmented generation, adversarial robustness of RAG and agent memory (retrieval/knowledge-base poisoning), language and script bias in multilingual retrieval, empirical evaluation and failure-mode analysis for LLM-based software, LLMs for software engineering and security (code generation, testing, sandbox analysis), and natural language processing.

My intended direction is the intersection of retrieval poisoning and multilingual retrieval bias: how adversarial content and cross-lingual/script disparities jointly degrade the trustworthiness of RAG systems, and methods and datasets to measure and mitigate both. This builds on the retrieval-evaluation and trustworthy-AI foundation below, which I aim to develop through doctoral training.

## Education

**George Mason University, Fairfax, Virginia** — Aug 2025 – May 2027 (Expected)
*M.S. in Software Engineering* — *GPA: 3.67 / 4.00*

**CVM University, India** — Aug 2020 – May 2024
*B.E. in Computer Engineering* — *CGPA: 9.04 / 10.00*
**Graduate Coursework:** Theory & Application of Data Mining (CS 584), Object-Oriented Software Specification & Construction (SWE 619), Software Modeling & Architecture Design (SWE 621), Software Project Management (SWE 625), User Interface Design & Development (SWE 632), Software Engineering for the Web (SWE 642), Software Testing (SWE 637, in progress), Advanced Natural Language Processing (CS 678, in progress)

## Professional Experience

**Software Engineer** — May 2024 – May 2025
*Ekaantik Software Solutions*
- Built a production real-time communication platform on Spring Boot, Kafka, Docker, and Kubernetes, supporting messaging and notification delivery at scale.
- Architected a WebSocket-based messaging engine with threading and delivery acknowledgement, sustaining sub-100 ms end-to-end latency.
- Managed Kubernetes manifests and container orchestration configurations, enabling zero-downtime deployment, CI pipelines, and horizontal autoscaling across distributed nodes.
- Implemented a RESTful API layer with token-based authentication, role-based access control, and request rate limiting across the user lifecycle.

**Software Engineering Intern** — Jan 2023 – Jun 2024
*Stat Modeller*
- Built FaceSense, a visitor-recognition system delivering real-time facial-recognition-based access control for enterprise deployment.
- Developed a Node.js REST API with paginated endpoints, event-driven logging, and real-time analytics sustaining sub-second response times.
- Engineered an OpenCV processing pipeline for live face detection and automated event triggering from IP camera streams.

## Research Projects

**Sandbox Escape Finder: Static + Dynamic Evaluation of RestrictedPython** | *Core, Follow-up* — *2026*
- Built a trustworthy-AI security tool that evaluates RestrictedPython's robustness against known Python sandbox-escape techniques, pairing AST-based static detection with a dynamic prober that executes payloads in an isolated subprocess. Stack: Python, RestrictedPython, pytest.
- Implemented AST pattern-matching detectors for five escape-technique classes (subclasses traversal, globals/closure access, builtins restoration, frame introspection, and format-string attribute access), each an independent detector sharing a single tree walk.
- Designed a three-signal escape oracle (canary-file diff, per-payload marker leak, and forbidden-module usage, combined with OR), correctly distinguishing parent-checkable from child-self-reported state across the subprocess boundary.
- Ran a 12-payload corpus (10 attack payloads + 2 benign controls) against RestrictedPython 8.5 with default guards, reporting zero successful escapes (8 blocked at compile time, 2 at runtime) and zero false negatives, and tied two payloads to CVE-2023-37271 and CVE-2023-41039; validated with a 17-test suite including seeded end-to-end reproducibility tests.
- Extended the work into executable proof-of-concept escapes to verify the oracle's signals actually fire, tracing the full pivot from a bare tuple through `object.__subclasses__()` to the live `os` module and independently confirming filesystem effects, secret leakage, and shell execution against a deliberately misconfigured sandbox.
- Built a 22-payload stress battery (dunder obfuscation via string concatenation and `chr()`, reflection builtins, format-string and frame-introspection variants) confirming correctly-configured RestrictedPython 8.5 blocks every payload, and grounded the thesis in an analysis of the Microsoft Copilot Studio Code Interpreter breach, where text-filter and AI-guardrail defenses failed.

**Course RAG Tutor: RAG Question Answering with a Measured Evaluation Harness** | *Independent* — *2025 – Present*
- Designed and implemented an end-to-end retrieval-augmented generation system over course PDFs: paragraph-aware chunking, Gemini embeddings, PostgreSQL/pgvector storage with HNSW indexing, and streaming, citation-grounded responses. Stack: Next.js, TypeScript, PostgreSQL/pgvector, Gemini.
- Built a 25-question evaluation harness measuring Precision@K, Recall@K, and Mean Reciprocal Rank (MRR = 0.888), enabling controlled, quantitative comparison of retrieval configurations rather than qualitative judgment of output quality.

- Diagnosed a chunking-induced retrieval failure in which a semantically distinct sentence was embedded together with unrelated boilerplate text, diluting its vector representation and causing retrieval to miss a directly relevant passage; identified paragraph-aware chunk boundaries as the root cause rather than the retrieval algorithm.
- Identified and corrected a validity failure in the evaluation set itself (date-format keyword mismatch), demonstrating that apparent system failures can originate in the measurement instrument rather than the system under test.
- Combined dense retrieval (pgvector cosine similarity) with lexical search (PostgreSQL full-text) via Reciprocal Rank Fusion to recover exact-term matches missed by embedding similarity alone.
- Diagnosed and resolved a duplicate-chunk index-contamination bug caused by non-idempotent ingestion, restoring correct top-ranked retrieval; introduced a filename/course uniqueness constraint to prevent recurrence.
- Built a chat-based interface with per-course conversation history and persistent storage, implementing token-level streaming responses via the Fetch API's ReadableStream body reader.
- Diagnosed a network-layer connection timeout between the application and its Postgres database, tracing it to broken IPv6 routing on the deployment network; resolved by forcing IPv4-first DNS resolution.

**Interview AI: Full-Stack Interview Preparation Platform** | *Independent Project* *2025*

- Developed a full-stack interview-preparation platform integrating document parsing, structured feedback generation, and progress tracking. Stack: React, Node.js, MongoDB, Gemini API.
- Engineered a voice-driven conversational interviewer using the Web Speech API for real-time spoken question answering and transcription.
- Integrated the Monaco editor with the Judge0 execution API to support in-browser code execution across three problem-difficulty tiers.

## Undergraduate Projects

**Emotion-Aware Music Recommendation System** | *Flask, OpenCV, CNN, Streamlit, Spotify API, NLTK*

- Built a web interface using HTML, CSS, and Flask to initiate the mood-detection workflow.
- Implemented real-time facial-expression analysis via webcam using a Convolutional Neural Network (CNN) with OpenCV for responsive mood detection.
- Developed a Streamlit application integrating the Spotify API to generate mood-based music recommendations, with adjustable threshold values to tune recommendation sensitivity.
- Applied NLTK-based polarity analysis to align song sentiment with detected mood, and included relevant YouTube video links alongside recommendations to boost user engagement.

**NLP-Based Drug Recommendation and Disease Prediction** | *Python, NLTK, scikit-learn*

- Cleaned and normalized a corpus of drug reviews using NLP preprocessing techniques, including lemmatization and stopword removal.
- Trained and compared Naive Bayes and Passive Aggressive classifiers for disease-condition classification from review text.
- Achieved 92.3% classification accuracy with the best-performing model.

**Loan Status Prediction** | *Pandas, NumPy, scikit-learn, Streamlit*

- Prepared the dataset by handling missing values, engineering features, and cleaning with Pandas and NumPy.
- Trained and compared five classification approaches logistic regression, random forest, decision tree, gradient boosting, and support vector machine (SVM) using scikit-learn.
- Selected the best-performing model by comparative evaluation and deployed it on Streamlit for interactive use.

**Twitter Data Analysis** | *Power BI*

- Conducted exploratory data analysis (EDA) on Twitter data to uncover trends, sentiment, and user-engagement patterns.
- Built interactive Power BI dashboards and reports to communicate insights effectively to a non-technical audience.

## Certifications & Online Coursework

- Analysis of Algorithms *(Princeton)*
- Introduction to Graph Theory *(UC San Diego)*
- Discrete Mathematics for CS *(Coursera)*
- Basic Statistics *(University of Amsterdam)*
- C Programming with Linux Specialization
- Core Java Specialization *(LearnQuest)*
- Cloud Computing & DevOps *(IBM)*
- Linux Commands & Shell Scripting *(IBM)*
- Introduction to SQL *(University of Michigan)*
- HTML5 & CSS3 Web Pages *(NIIT)*

## Technical Skills

**Programming Languages:** Python, Java, C, C++, JavaScript/TypeScript.
**LLM & Retrieval Systems:** Retrieval-augmented generation, hybrid dense/lexical retrieval (RRF), embedding-based semantic search, retrieval evaluation (Precision@K, Recall@K, MRR), citation-grounded generation.
**Program Analysis & Security:** AST-based static analysis, dynamic analysis and probing, Python sandbox-escape analysis, RestrictedPython, subprocess isolation.
**Testing & Evaluation:** pytest, unit and end-to-end testing, evaluation-harness design, reproducible (seeded) experiments.
**Distributed Systems & Backend:** Kafka, WebSockets, Kubernetes, Docker, Spring Boot, Node.js, REST APIs, microservices.
**Data & Storage:** PostgreSQL/pgvector, MongoDB, MySQL, HNSW indexing.
**Cloud & Infrastructure:** AWS (EC2, S3), Microsoft Azure, CI/CD.
**Computer Vision:** OpenCV.
**Tools**: Git, Linux, Postman.